2: A splash-screen boot sector

Due by noon, Sept 11, 2023 (Mon)

In this homework, we use BIOS services to create a "splash screen" boot sector. This requires:

- Creating a disk image that contains the boot sector and the picture to be rendered,
- Waiting for the user to press a key, and outputting text on the console.
- changing the video mode and palette settings,
- reading an image from the disk into memory,
- (bonus points) manipulate palette settings.

Before you start: this page (Booting a PC) provides useful information related to this homework:

- "Getting Started with x86 assembly" provides a few useful links for writing x86 assembly code.
- "The PC's Physical Address Space" shows the layout of the physical address layout at boot time (the *Real Mode*).
- "Part 2: The Boot Loader" discusses the boot routine.

Intel 8086: The assembly code you're going to write is for this very old CPU model. Today's x86 CPUs can still run these code because of backward compatibility!

You will write assembly code in AT&T syntax. (AT&T Syntax versus Intel Syntax)

You only need to know a minimal set of 8086 assembly to get this homework done. The following are the references:

- The x86 assembly wikibooks
- x86 Assembly Guide
- All of the *.S source files in the xv6 directory
- Comprehensive BIOS documentation (PDF)

Intel 8086 microprocessor registers

Getting started

The template for this homework is in the class repository, **https://github.com/CS461/xv6-public.git**, under the hw2 branch. If you haven't already cloned the xv6-public repository, do so first, then **git checkout origin/hw2 -b hw2** to get the template.

Create your submission repository using this URL: **https://classroom.github.com/a/Pa94xvZx**. As in hw1, add the hw2 submission repository, and use git push to submit your work.

A skeleton boot sector is provided in **bootskel.S** in this repository.

The default bootsplash image "**cover.raw**" is in the same folder. "cover.raw" contains the image data that can be directly used for video mode 13H.

Optional: If you want, use your own image instead! The image should be 320x200 pixels, with each pixel represented by one byte: a palette index. The palette can be modified, but easiest will be to use the standard VGA palette, following these instructions.

Try the demo code:

- $ make bootskel.img
- $ qemu-system-i386 bootskel.img
- An "H" appears on the screen.

Note the qemu-system-i386 here. To run the boot sector, x86_64 works just as well, but to use the debugger, the i386 appears to be the better choice.

study tip: how does i8086 assembly differ from the 64-bit assembly we're used to? What does the 16-bit/64-bit actually refer to? How does this limit what programs can be written, or what hardware can be supported?

In depth: besides the change from 16, to 64 bits, what other major differences exist between this 1978 processor, and a modern CPU? Think about things beyond speed.

Creating the disk

Create a bootsplash.S using bootskel.S as the template. `cp bootskel.S bootsplash.S`. You're going to write code in `bootsplash.S`.

Copy commands from the `bootskel.img` target to create a new `bootsplash.img` target in the *Makefile*.

Now the bootsplash.img only contains the bootsector. We need to add the picture file (cover.raw) to the disk image so that it can be accessed at boot time.

Use dd to create the full disk image by appending cover.raw to the boot sector you just created.

dd arguments:

- **`if=`(input file)**
- **`of=`(output file)**
- `bs=`(block size, default: 512B)
- `count=`(# of blocks, default: copy everything)
- `skip=`(skip input blocks, default: 0)
- **`seek=`(write to an offset in the output, default: 0)**

You don't need to specify an argument if the default value is what you need.

Use `xxd` to visually inspect the bootsplash.img. How to verify that it contains the correct data?

```
xxd bootsplash.img | less
```

You should see the boot sector code comes first, followed by the cover.raw's data after the "0x55 0xaa".

Once done, try your finished disk by running

```
qemu-system-i386 bootsplash.img
```

You should see a single character `H` appear on the screen because we have not added any code to the assembly.

Follow the instructions below to implement the bootsplash.S. You're going to write 10–20 lines of code.

study tip: try using objdump with the -S option, to read bootsplash.img in a different way. You'll see some familiar instructions first. What are all the other instructions that follow? What about the stuff after the 0xaa55? Speaking of 0xaa55. Sometimes it's ar first, then 55. Other times it's 55 first, then aa. Make sure you understand little-endian and big-endian byte order, and what it implies for what a number looks like in memory. Maybe ask chatgpt for help. Does it apply to things other than numbers? What about character arrays? Pointers?

In depth: What is this dd program anyway, and what's it really for? What's the difference between a disk image and a disk? You've heard of formatting a disk. Can you format a disk image too?

Writing text to the screen and waiting for keypress

When in text mode, you can use `Int 10h, AH=0Eh` to write a single character at a time to the screen, and use a loop to print a full string. If you do it this way, make sure you keep your iterator in a place that isn't overwritten by your code (or the BIOS interrupt routine). One such good place would be a byte somewhere in memory.

You may also use `Int 10h, AH=13h` to print a string directly. This one, however, requires you to specify a cursor position. To avoid hard-coding the position, you'll have to use a different BIOS routine to see where the cursor is now.

Hint: keep in mind that you'll want the address of your string in `%bp` not the first character or two of the string.

`Int 10h, AH=13h`: **%bl controls the color**. You need a non-zero value in `%bl` to make it visible. the highest 4 bits: background; The lowest 4 bits: foreground. With `0x0f` fg=white, bg=black.

Update the boot sector code to say "`Welcome to xv6@UIC. Press any key to continue`" instead of "H". Then wait for a keypress before continuing. Use `Int 16h, AH=00` to wait for a keypress.

debugging tip: Sometimes, it can be really handy to see what the end result looks like, rather than the assembly you wrote. For this, use the disassembly function of **objdump**. In order to disassemble i8086 assembly, you need to be a little extra specific. Try the following (to disassemble objectfile.o):

```
objdump -D -mi386 -Maddr16,data16 objectfile.o
```

Other times, you might want to inspect the contents of registers while your program is running. For this, you can actually attach gdb to qemu. Start qemu with added "-S -s" parameters, and qemu will wait for gdb to attach. Then fire up gdb, and use the command "target remote localhost:1234" to attach. Now you can use gdb as normal. For example, `break *0x7c00` and `continue` will bring you to the first instruction of your boot sector.

study tip: Two different ways to implement the functionality above were provided. See if you can do both.

In depth: Where do these interrupts come from? Can you figure out how they were implemented? If you run your program with gdb, can you find the code that implements the interrupt handler? The interrupt vector is stored starting at address 0. Each entry is 32 bits, pointer and segment. What would be the address of the entry for interrupt 0x10? (Note: disassembly with gdb's x/i is not reliable for 16-bit code).

Changing video mode

BIOS interrupt `10h` lets you set the video mode using function `AH=00h`. Use video mode `13h` for this homework.

You should notice the screen going blank (and perhaps the emulator window changing shape) upon success.

Once in video mode, the screen will show whatever is in memory starting at address 0xA0000. However, the video buffer is automatically cleared by BIOS when switching modes.

study tip: what do we mean by "show what is in memory" here? How are pixels represented in this video mode? What about other video modes? How is a pixel represented in a modern video buffer?

Writing graphics to the screen

Once you put something other than zeroes in 0xA0000, you should notice something on the screen once you are in video mode `13h`.

How large is the video buffer in this video mode?

Note that on the 16-bit mode, the general-purpose registers can only help you address below 64KB:

```
mov $0xfffe, %ax
mov (%ax), %bx
```

To reach an address above `0xA0000`, you need the (`base * 16 + offset`) addressing:

```
mov $0xA000, %ax  # How many 0s should it have?
mov %ax, %ds  # In 16BIG you cannot move imm directly to a *s register. (mov %0xA000,%ds)
mov$ $0xf, %al # A white pixel
mov $0, %di   # The first pixel at the top-left corner of the screen
movb %al, %ds:(%di) # After this instruction you can see the tiny white dot there. (maximize window to make it more visible)
```

Then put the image from the disk onto the screen, as per below.

Extended reading: http://mal.vonos.net/linux/graphics-stack/

study tip: segmented addressing was introduced as a band-aid in 8086. Understand how to use it, what it's for, and the syntax for segmented addressing.

in depth: data and code segments aren't encountered much in modern 64-bit programs. However, the FS segment register is more important than ever. What's the FS segment used for in a modern program?

Reading from disk using BIOS services

`Int 13h`, function `AH=42h` reads from disk.

- Fill in a disk address packet structure, specifying the buffer address, start sector, number of sectors to read, etc.. (hint: You can hard-code this structure)
- Call the interrupt to get the buffer filled in.
- Note that the "transfer buffer" address is a 32-bit field, in segment:offset format. Pay attention to byte order.

The term **block** is used interchangeably with **sector**, which is 512 bytes.

We need to use the GNU syntax to declare the values stored in the boot sector:

- ".byte" for BYTE values (and strings)
- ".word" for WORD values (2-byte)
- ".long" for DWORD values (4-byte)
- ".quad" for QWORD values (8-Byte)

Hint: Read this link! "segment:offset pointer to the memory buffer to which sectors will be transferred (note that x86 is little-endian)' if declaring the segment and offset separately, the offset must be declared before the segment)". It's convenient to use two ".word" values instead of one ".long".

Hint: the boot drive number can be obtained from %dl register upon the start of boot sector code. You need to save the value before it gets erased from the register. More information can be found here (the "technical details" section).

To see if you succeed, start qemu with the flags "`-s -S`" to have it wait for gdb. Start gdb in a separate Linux window, and type `target remote 127.0.0.1:1234` if running qemu in Linux. If running outside Linux, replace 127.0.0.1 with the IP address of your host machine.

Now you can inspect your memory.

- `x/10buwx 0x7c00` prints out 100 unsigned words in hexadecimal, starting at 0x7c00. This should show part of your boot sector binary.
- `x/101 0x7c00` shows 10 **instructions** starting at 0x7c00. Very helpful! It's worth noting, however, that in recent versions of gdb, support for 16-bit mode is decidedly poor. You may see your instructions incorrectly interpreted as 32-bit mode instructions, which can be confusing.

To verify if you have correctly loaded the image into the memory, you can compare the values with that in the cover.raw file: `xxd cover.raw`

Note: the bootasm.S and boottmain.c files that make up the xv6 boot sector do not use BIOS services to read from disk. Instead, they directly access the disk using the **in/out/x** instructions. Both are legitimate ways to access the disk, but for this assignment, use the BIOS services.

study tip: experiment with x/i, x/uwx and other ways to print the same data in memory. Can you tell what is probably code, and what is probably data, or something else? You could try x/20i $rip, x/100uwx $rip, x/20i $rsp, x/100uwx $rsp, for example.

in depth: in general, given some bytes, how can you (can you?) tell what is supposed to be? In Java and Python and Javascript, every object starts with a pointer indicating its type. In low level languages, like assembly, C and Rust, there's no such pointer.

(Bonus task) Manipulate palette settings

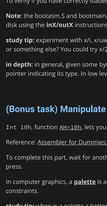
`Int 10h`, function `AH=10h`, lets you configure the palette.

Reference: Assembler for Dummies: Graphics II – palette. (Beware that this post uses the Intel syntax.)

To complete this part, wait for another key press after displaying the initial image. Then change some of the colors in the image and wait for another key press.

In computer graphics, a **palette** is a finite set of colors. Palettes can be optimized to improve image accuracy in the presence of software or hardware constraints.

study tip: when is a palette a better choice than directly encoding the color in the data, RGB or CMYK fashion? When is a palette harder or easier to use?



Submit your code

Commit your changes locally. You can double check the local commits with "git log" or "tig".

Since you cloned your repository from Github, you should see one remote named "origin" with command "git remote -v".

In this scenario, the local branch "master" should have been set to track remote branch "master" in "origin".

To push your local commits to Github, enter "git push".

Alternatively, you can also manually specify the remote and branch names following the instructions in hw1.

Once this is done, you should be able to see your commit(s) in a browser at URL "https://github.com/CS461/hw2-YOUR-GITHUB-ID".